

HW03

ENAE 601: Astrodynamics

October 07, 2026 at 23:59:00

Dr. Healy

Vai Srivastava

Problem 1

Curtis 4.3

For a geocentric satellite whose inertial position and velocity vectors in a geocentric equatorial frame are:

$$\mathbf{r} = 2500\hat{\mathbf{I}} + 16000\hat{\mathbf{J}} + 4000\hat{\mathbf{K}} \quad (\text{km})$$

$$\mathbf{v} = -3\hat{\mathbf{I}} - \hat{\mathbf{J}} + 5\hat{\mathbf{K}} \quad (\text{km s}^{-1})$$

- Find the classical orbital elements e , a , i , Ω , ω , θ .
- Compute the equinoctial orbital elements a , h (*this is not angular momentum*), k , p (*this is not semilatus rectum*), q , λ .

- To distinguish angular momentum from the equinoctial element h , define:

$$\begin{aligned}\mathbf{H} &= \mathbf{r} \times \mathbf{v} \\ H &= |\mathbf{H}|.\end{aligned}$$

The node vector, eccentricity vector, and specific energy are:

$$\begin{aligned}\mathbf{N} &= \hat{\mathbf{K}} \times \mathbf{H}, \\ \mathbf{e} &= \frac{\mathbf{v} \times \mathbf{H}}{\mu} - \frac{\mathbf{r}}{r}, \\ \mathcal{E} &= \frac{v^2}{2} - \frac{\mu}{r}, \\ a &= -\frac{\mu}{2\mathcal{E}}.\end{aligned}$$

Substituting the given state gives:

$$\begin{aligned}\mathbf{H} &= [84000, -24500, 45500] \text{ (km}^2 \text{ s}^{-1}\text{)}, \\ \mathbf{N} &= [24500, 84000, 0] \text{ (km}^2 \text{ s}^{-1}\text{)}, \\ \mathbf{e} &= [0.0433034541, 0.4369514387, 0.1553367056], \\ \mathcal{E} &= -6.395\,697\,282 \text{ km}^2 \text{ s}^{-2}.\end{aligned}$$

To retain the correct quadrants, the oriented angle is:

$$A(\mathbf{x}, \mathbf{y}) = \text{atan2}(\hat{\mathbf{H}} \cdot (\mathbf{x} \times \mathbf{y}), \mathbf{x} \cdot \mathbf{y}), \quad \hat{\mathbf{H}} = \frac{\mathbf{H}}{H}.$$

Reduce angles mod 2π into $[0, 2\pi)$. Thus:

$$\begin{aligned}i &= \arccos\left(\frac{H_z}{H}\right), \\ \Omega &= \text{atan2}(N_y, N_x), \\ \omega &= A(\mathbf{N}, \mathbf{e}), \\ \theta &= A(\mathbf{e}, \mathbf{r}).\end{aligned}$$

Here $v_r = \frac{\mathbf{r} \cdot \mathbf{v}}{r} = -0.209\,822 \text{ km s}^{-1} < 0$, so the satellite is approaching perigee and $\theta > 180^\circ$.

Thus, the classical orbital elements are:

$$\begin{aligned}e &= 0.4657587799, \\a &= 31161.574\,916 \text{ km}, \\i &= 62.525\,568^\circ, \\\Omega &= 73.739\,795^\circ, \\\omega &= 22.080\,536^\circ, \\\theta &= 353.600\,347^\circ \quad \blacksquare\end{aligned}$$

b) Use the equinoctial convention:

$$\begin{aligned}h &= e \sin(\Omega + \omega), \\k &= e \cos(\Omega + \omega), \\p &= \tan\left(\frac{i}{2}\right) \sin(\Omega), \\q &= \tan\left(\frac{i}{2}\right) \cos(\Omega), \\\lambda &= \Omega + \omega + M.\end{aligned}$$

Here a is unchanged, h, k, p, q are dimensionless, and λ is mean longitude.

Obtain the mean anomaly from:

$$\begin{aligned}E &= \text{atan2}\left(\sqrt{1 - e^2} \sin(\theta), e + \cos(\theta)\right), \\M &= E - e \sin(E),\end{aligned}$$

reducing E , M , and λ via $\text{mod } 2\pi$.

Noting that all anomaly calculations use radians, substitution and conversion gives:

$$\begin{aligned}E &= 356.133\,830^\circ \\M &= 357.933\,167^\circ.\end{aligned}$$

Thus, the equinoctial orbital elements are:

$$\begin{aligned}a &= 31161.574\,916 \text{ km}, \\h &= 0.4633576943, \\k &= -0.0472322793, \\p &= 0.5828354153, \\q &= 0.1699936628, \\\lambda &= 93.753\,498^\circ \quad \blacksquare\end{aligned}$$

Problem 2

Curtis 4.4

At a given instant, the position $\mathbf{r}$ and velocity $\mathbf{v}$ of a satellite in the geocentric equatorial frame are:

$$\mathbf{r} = -13000\hat{\mathbf{K}} \quad (\text{km})$$

$$\mathbf{v} = 4\hat{\mathbf{I}} + 5\hat{\mathbf{J}} + 6\hat{\mathbf{K}} \quad (\text{km s}^{-1})$$

- Find the classical orbital elements $e, a, i, \Omega, \omega, \theta$.
- Compute the equinoctial orbital elements a, h (*this is not angular momentum*), k, p (*this is not semilatus rectum*), q, λ .

- Using the state-to-elements relations from Problem 1, Part A,

$$\mathbf{H} = [65000, -52000, 0] \quad (\text{km}^2 \text{s}^{-1}),$$

$$\mathbf{N} = [52000, 65000, 0] \quad (\text{km}^2 \text{s}^{-1}),$$

$$\mathbf{e} = [0.7827395886, 0.9784244857, -0.3371801305],$$

$$\mathcal{E} = 7.838\,461\,538 \text{ km}^2 \text{s}^{-2}.$$

The positive specific energy and $e > 1$ identify a hyperbola, with signed semimajor axis $a = -\frac{\mu}{2\mathcal{E}} < 0$.

Since $H_z = 0$, the orbit is polar: $i = 90^\circ$. Although $v_z > 0$, the satellite is below the equatorial plane and moving inward:

$$v_r = \frac{\mathbf{r} \cdot \mathbf{v}}{r} = -6.000\,000 \text{ km s}^{-1} < 0.$$

Thus, the signed true anomaly is negative. Evaluating the oriented angles and reporting θ in $[0, 2\pi)$ gives:

$$e = 1.2975693346,$$

$$a = -25\,425.907\,753 \text{ km},$$

$$i = 90.000\,000^\circ,$$

$$\Omega = 51.340\,192^\circ,$$

$$\omega = 344.938\,530^\circ,$$

$$\theta = 285.061\,470^\circ \quad \blacksquare$$

Equivalently, $\theta = -74.938\,530^\circ$ on the physical hyperbolic branch, $-\arccos(-\frac{1}{e}) < \theta < \arccos(-\frac{1}{e})$, consistent with approach to perigee.

- The definitions of h, k, p, q in Problem 1, Part B still describe the eccentricity vector and orbital plane. With $\tan(\frac{i}{2}) = 1$:

$$p = \sin(\Omega) = \frac{5}{\sqrt{41}},$$

$$q = \cos(\Omega) = \frac{4}{\sqrt{41}}.$$

The elliptic eccentric anomaly E and the formulas involving $\sqrt{1 - e^2}$ do not apply. Instead, use the signed hyperbolic eccentric anomaly F_h and hyperbolic mean anomaly M_h :

$$\begin{aligned}
F_h &= \operatorname{arsinh} \left(\frac{\sqrt{e^2 - 1} \sin(\theta)}{1 + e \cos(\theta)} \right) \\
&= -0.5663484751, \\
M_h &= e \sinh(F_h) - F_h = -0.2084481530.
\end{aligned}$$

These are dimensionless, unbounded quantities, meaning both are negative before perigee. In particular, $M_h = \sqrt{\frac{\mu}{(-a)^3}}(t - t_p)$, where t_p is the time of perigee passage.

To retain the requested mean-longitude element, explicitly extend its definition to the hyperbola:

$$\begin{aligned}
\varpi &= (\Omega + \omega) \bmod 2\pi \\
&= 36.278\,722^\circ, \\
\lambda &= \varpi + M_h.
\end{aligned}$$

Here ϖ is fixed in $[0, 2\pi)$ and converted to radians before adding M_h . Neither M_h nor λ is reduced $\bmod 2\pi$, as doing so would lose the position along the nonperiodic trajectory. This is a hyperbolic extension of the equinoctial elements, so the elliptic eccentric-longitude equations used for the other problems are unfortunately inapplicable.

With this convention, the elements are:

$$\begin{aligned}
a &= -25\,425.907\,753 \text{ km}, \\
h &= 0.7677897281, \\
k &= 1.0460330356, \\
p &= 0.7808688094, \\
q &= 0.6246950476, \\
\lambda &= 24.335\,522^\circ \quad \blacksquare
\end{aligned}$$

As a check, solving $e \sinh(F_h) - F_h = \lambda - \varpi$ and reconstructing the inertial state recovers $\mathbf{r} = [0, 0, -13000] \text{ km}$ and $\mathbf{v} = [4, 5, 6] \text{ km s}^{-1}$ to numerical precision (an implementation of this can be seen in the Python code for this problem).

Problem 3

Curtis 4.5

At time t_0 (relative to perigee passage) the position $\mathbf{r}$ and velocity $\mathbf{v}$ of a satellite in the geocentric equatorial frame are:

$$\mathbf{r} = 6500\hat{\mathbf{I}} - 7500\hat{\mathbf{J}} - 2500\hat{\mathbf{K}} \quad (\text{km})$$

$$\mathbf{v} = 4\hat{\mathbf{I}} + 3\hat{\mathbf{J}} - 3\hat{\mathbf{K}} \quad (\text{km s}^{-1})$$

- Find the classical orbital elements e , a , i , Ω , ω , θ .
- Compute the equinoctial orbital elements a , h (*this is not angular momentum*), k , p (*this is not semilatus rectum*), q , λ .

- The angular momentum, node vector, eccentricity vector, and energy are:

$$\mathbf{H} = [30000, 9500, 49500] \text{ (km}^2 \text{ s}^{-1}\text{)},$$

$$\mathbf{N} = [-9500, 30000, 0] \text{ (km}^2 \text{ s}^{-1}\text{)},$$

$$\mathbf{e} = [-0.1910373697, 0.0102691021, 0.1138093863],$$

$$\mathcal{E} = -21.945\,768\,548 \text{ km}^2 \text{ s}^{-2}.$$

Here $N_x < 0$ and $N_y > 0$, so Ω lies in the second quadrant.

Also, $v_r = 1.074\,770 \text{ km s}^{-1} > 0$, so the satellite is moving away from perigee.

Using the relations in Problem 1, Part A gives:

$$e = 0.2226057220,$$

$$a = 9081.477\,350 \text{ km},$$

$$i = 32.445\,017^\circ,$$

$$\Omega = 107.571\,259^\circ,$$

$$\omega = 72.358\,601^\circ,$$

$$\theta = 134.725\,887^\circ \quad \blacksquare$$

- From the true anomaly and eccentricity and the relations in Problem 1, Part B, we obtain:

$$E = 124.783\,408^\circ,$$

$$M = 114.308\,061^\circ.$$

The longitude of perigee is $\Omega + \omega = 179.929\,860^\circ$.

Thus, applying the equinoctial definitions gives:

$$a = 9081.477\,350 \text{ km},$$

$$h = 0.0002725101,$$

$$k = -0.2226055552,$$

$$p = 0.2773776977,$$

$$q = -0.0878362709,$$

$$\lambda = 294.237\,920^\circ \quad \blacksquare$$

Problem 4

Curtis 4.6

With respect to the geocentric equatorial frame, the position vector of a spacecraft is $\mathbf{r} = -6000\hat{\mathbf{I}} - 1000\hat{\mathbf{J}} - 5000\hat{\mathbf{K}}$ (km) and the orbit's eccentricity vector is $\mathbf{e} = 0.4\hat{\mathbf{I}} + 0.5\hat{\mathbf{J}} + 0.6\hat{\mathbf{K}}$. Calculate the true anomaly θ if the satellite is approaching perigee.

The eccentricity vector points toward perigee, so:

$$\cos(\theta) = \frac{\mathbf{e} \cdot \mathbf{r}}{er}.$$

The required magnitudes and dot product are:

$$\begin{aligned} e &= \sqrt{0.4^2 + 0.5^2 + 0.6^2} = 0.8774964387, \\ r &= \sqrt{6000^2 + 1000^2 + 5000^2} \text{ km} = 7874.007874 \text{ km}, \\ \mathbf{e} \cdot \mathbf{r} &= -5900 \text{ km}. \end{aligned}$$

Thus, we have:

$$\begin{aligned} \cos(\theta) &= -0.8539074533, \\ \arccos(\cos(\theta)) &= 148.639244^\circ. \end{aligned}$$

Unfortunately, the inverse cosine alone does not distinguish the inbound and outbound branches.

Since $v_r = \left(\frac{\mu}{H}\right)e \sin(\theta) < 0$ when approaching perigee, θ is bounded by $180^\circ < \theta < 360^\circ$.

Therefore, the true anomaly is:

$$\begin{aligned} \theta &= 360^\circ - 148.639244^\circ \\ &= 211.360756^\circ \quad \blacksquare \end{aligned}$$

Problem 5**Curtis 4.7**

Given that, relative to the geocentric equatorial frame, $\mathbf{r} = -6600\hat{\mathbf{I}} - 1300\hat{\mathbf{J}} - 5200\hat{\mathbf{K}}$ (km), the eccentricity vector is $\mathbf{e} = -0.4\hat{\mathbf{I}} - 0.5\hat{\mathbf{J}} - 0.6\hat{\mathbf{K}}$, and the satellite is flying toward perigee, calculate the inclination of the orbit.

Both $\mathbf{r}$ and $\mathbf{e}$ lie in the orbital plane, so their cross product determines the normal up to its sign.

With θ measured in the direction of motion, the cross product is:

$$\mathbf{e} \times \mathbf{r} = er \sin(\theta) \hat{\mathbf{H}}.$$

The satellite is flying toward perigee, so $\sin(\theta) < 0$.

Therefore, the unit angular momentum vector is:

$$\hat{\mathbf{H}} = \frac{\mathbf{r} \times \mathbf{e}}{|\mathbf{r} \times \mathbf{e}|}.$$

Computing the cross product gives:

$$\begin{aligned} \mathbf{r} \times \mathbf{e} &= \det \left(\begin{bmatrix} \hat{\mathbf{I}} & \hat{\mathbf{J}} & \hat{\mathbf{K}} \\ -6600 & -1300 & -5200 \\ -0.4 & -0.5 & -0.6 \end{bmatrix} \right) \\ &= [-1820, -1880, 2780] \text{ (km)}, \\ |\mathbf{r} \times \mathbf{e}| &= 3817.748 \text{ 027 km}. \end{aligned}$$

The inclination is the angle between $\hat{\mathbf{H}}$ and $\hat{\mathbf{K}}$:

$$\cos(i) = \hat{\mathbf{H}} \cdot \hat{\mathbf{K}} = \frac{2780}{\sqrt{1820^2 + 1880^2 + 2780^2}} = 0.7281779678.$$

Thus, the inclination of the orbit is:

$$i = \arccos(0.7281779678) = 43.266 \text{ 137}^\circ \quad \blacksquare$$

Problem 6

Find the equinoctial elements for the following orbits:

a) Position and Velocity:

$$\mathbf{r} = \begin{bmatrix} 3698.3455 \\ -34232.4473 \\ 0 \end{bmatrix} \text{ (km)}, \quad \mathbf{v} = \begin{bmatrix} 2.1482 \\ -1.4944 \\ 0 \end{bmatrix} \text{ (km s}^{-1}\text{)}$$

b) Position and Velocity:

$$\mathbf{r} = \begin{bmatrix} -21082.0848 \\ 36515.242 \\ 0 \end{bmatrix} \text{ (km)}, \quad \mathbf{v} = \begin{bmatrix} -2.6627 \\ -1.5373 \\ 0 \end{bmatrix} \text{ (km s}^{-1}\text{)}$$

c) Position and Velocity:

$$\mathbf{r} = \begin{bmatrix} -15843.4562 \\ -2247.7766 \\ 21200.4627 \end{bmatrix} \text{ (km)}, \quad \mathbf{v} = \begin{bmatrix} 1.4544 \\ -3.5188 \\ 0.7138 \end{bmatrix} \text{ (km s}^{-1}\text{)}$$

a) The orbit is equatorial and prograde because $H_x = H_y = 0$ and $H_z > 0$.

Thus, $i = 0$ and $p = q = 0$.

Although the ascending node is undefined, the longitude of perigee $\varpi = \Omega + \omega$ is well defined for this eccentric orbit.

In the equatorial plane, the eccentricity vector gives $k = e_x$ and $h = e_y$:

$$\mathbf{e} = [-0.3623940235, 0.6276769176, 0],$$

$$e = 0.7247811677.$$

We can obtain the true longitude directly from position via:

$$L = \theta + \omega + \Omega = \text{atan2}(r_y, r_x) = 276.166\,106^\circ.$$

We now require the equinoctial eccentric longitude and the parameter β :

$$F = \varpi + E$$

$$\beta = \frac{1}{1 + \sqrt{1 - h^2 - k^2}}.$$

With $\frac{1}{\beta} = 1 + \sqrt{1 - h^2 - k^2}$, the eccentric longitude is:

$$F = L + 2 \text{atan2}\left(h \cos(L) - k \sin(L), \frac{1}{\beta} + k \cos(L) + h \sin(L)\right).$$

Recall that $\lambda = \varpi + E - e \sin(E)$, meaning:

$$\lambda = F + h \cos(F) - k \sin(F).$$

After reducing $\lambda \bmod 2\pi$, the equinoctial elements are:

$$\begin{aligned}
a &= 24446.301\,604 \text{ km}, \\
h &= 0.6276769176, \\
k &= -0.3623940235, \\
p &= 0, \\
q &= 0, \\
\lambda &= 209.998\,682^\circ \quad \blacksquare
\end{aligned}$$

b) Again, $H_x = H_y = 0$ and $H_z > 0$, so $p = q = 0$, $h = e_y$, and $k = e_x$.

Computing directly from the given state gives:

$$\begin{aligned}
e &= 0.0000278396 \\
L &= 120.000\,000^\circ.
\end{aligned}$$

This orbit is nearly circular. Retaining the small eccentricity from the supplied rounded state, and using the longitude equations, gives:

$$\begin{aligned}
a &= 42163.002\,517 \text{ km}, \\
h &= -0.0000224843, \\
k &= 0.0000164164, \\
p &= 0, \\
q &= 0, \\
\lambda &= 119.999\,659^\circ \quad \blacksquare
\end{aligned}$$

In the circular limit, $h = k = 0$ and $\lambda = L$, so no perigee direction is needed.

c) For an inclined orbit, we obtain p and q directly from angular momentum:

$$p = \frac{H_x}{H + H_z}, \quad q = -\frac{H_y}{H + H_z}.$$

Let $s = 1 + p^2 + q^2$ and introduce the orthonormal axes in the orbital plane

$$\begin{aligned}
\hat{\mathbf{f}} &= \frac{1}{s}[1 - p^2 + q^2, 2pq, -2p], \\
\hat{\mathbf{g}} &= \frac{1}{s}[2pq, 1 + p^2 - q^2, 2q].
\end{aligned}$$

These reduce to $\hat{\mathbf{I}}$ and $\hat{\mathbf{J}}$ when $p = q = 0$. In this basis,

$$\begin{aligned}
k &= \mathbf{e} \cdot \hat{\mathbf{f}}, \\
h &= \mathbf{e} \cdot \hat{\mathbf{g}}, \\
L &= \text{atan2}(\mathbf{r} \cdot \hat{\mathbf{g}}, \mathbf{r} \cdot \hat{\mathbf{f}}).
\end{aligned}$$

Substitution gives:

$$\begin{aligned}
\mathbf{H} &= [72995.725212, 42143.011986, 59019.119964] \text{ (km}^2 \text{ s}^{-1}\text{)}, \\
e &= 0.0000143440, \\
L &= 196.999\,726^\circ.
\end{aligned}$$

Using the longitude equations and the energy relation for a , the equinoctial elements are:

$$\begin{aligned}a &= 26562.132\,167\text{ km}, \\h &= -0.0000073796, \\k &= -0.0000123001, \\p &= 0.4508260379, \\q &= -0.2602778048, \\\lambda &= 197.000\,123^\circ \quad \blacksquare\end{aligned}$$

This orbit is also nearly circular. The small nonzero values of h and k are retained rather than setting them to zero.

Code

```

./src/index.py Python
1  from pathlib import Path
2
3  import numpy as np
4  from scipy.optimize import brentq
5
6  MU_EARTH = 398600.0 # km^3/s^2
7  OUTPUT_DIR = Path(__file__).resolve().parents[1] / "outputs" / "text"
8  TAU = 2.0 * np.pi
9
10
11 def heading(problem: str) -> None:
12     """Print the heading used to separate each problem's output."""
13     print(f"\n-----\n {problem}\n-----")
14
15
16 def write_output(problem: str, lines: list[str]) -> None:
17     """Print and save a problem's text output for inclusion in Typst."""
18     output = "\n".join(lines) + "\n"
19     print(output, end="")
20     OUTPUT_DIR.mkdir(parents=True, exist_ok=True)
21     (OUTPUT_DIR / f"s{problem}.txt").write_text(output, encoding="utf-8")
22
23
24 def angle(first: np.ndarray, second: np.ndarray, normal: np.ndarray) -> float:
25     """Return the oriented angle about a unit normal in [0, 2 pi)."""
26     return np.arctan2(np.dot(normal, np.cross(first, second)), first @ second) % TAU
27
28
29 def geometry(r: np.ndarray, v: np.ndarray) -> dict:
30     """Compute two-body invariants from Cartesian position and velocity."""
31     radius = np.linalg.norm(r)
32     momentum = np.cross(r, v)
33     eccentricity = np.cross(v, momentum) / MU_EARTH - r / radius
34     energy = v @ v / 2.0 - MU_EARTH / radius
35     return {
36         "r": radius,
37         "vr": r @ v / radius,
38         "H": momentum,
39         "N": np.cross([0.0, 0.0, 1.0], momentum),
40         "ev": eccentricity,
41         "e": np.linalg.norm(eccentricity),
42         "a": -MU_EARTH / (2.0 * energy),
43         "energy": energy,
44     }

```

```

45
46
47 def classical(g: dict, r: np.ndarray) -> dict:
48     """Classical elements for noncircular, inclined ellipses and hyperbolas."""
49     momentum, node, eccentricity = g["H"], g["N"], g["ev"]
50     if np.linalg.norm(node) < 1e-12 or g["e"] < 1e-12:
51         raise ValueError("Classical node/perigee angles are undefined.")
52     normal = momentum / np.linalg.norm(momentum)
53     theta = angle(eccentricity, r, normal)
54     elements = {
55         "e": g["e"],
56         "a": g["a"],
57         "i": np.arctan2(np.linalg.norm(momentum[:2]), momentum[2]),
58         "Omega": np.arctan2(node[1], node[0]) % TAU,
59         "omega": angle(node, eccentricity, normal),
60         "theta": theta,
61     }
62     if g["e"] < 1.0:
63         anomaly = (
64             np.arctan2(
65                 np.sqrt(1.0 - g["e"] ** 2) * np.sin(theta), g["e"] + np.cos(theta)
66             )
67             % TAU
68         )
69         elements.update(E=anomaly, M=(anomaly - g["e"] * np.sin(anomaly)) % TAU)
70     elif g["e"] > 1.0:
71         signed_theta = np.arctan2(np.sin(theta), np.cos(theta))
72         anomaly = hyperbolic_anomaly(g["e"], signed_theta)
73         elements.update(
74             theta_signed=signed_theta,
75             F=anomaly,
76             M_h=g["e"] * np.sinh(anomaly) - anomaly,
77             varpi=(elements["Omega"] + elements["omega"]) % TAU,
78         )
79     else:
80         raise ValueError("Parabolic orbits require a different element set.")
81     return elements
82
83
84 def hyperbolic_anomaly(e: float, theta: float) -> float:
85     """Signed hyperbolic anomaly F; inbound values are negative, never wrapped."""
86     denominator = 1.0 + e * np.cos(theta)
87     if e <= 1.0 or denominator <= 0.0:
88         raise ValueError(
89             "A true anomaly on the physical hyperbolic branch is required."
90         )

```

```

91     return np.arcsinh(np.sqrt(e * e - 1.0) * np.sin(theta) / denominator)
92
93
94 def equinoctial_basis(p: float, q: float) -> tuple[np.ndarray, np.ndarray]:
95     """Orthonormal in-plane axes, nonsingular at i = 0."""
96     scale = 1.0 + p * p + q * q
97     f = np.array([1.0 - p * p + q * q, 2.0 * p * q, -2.0 * p]) / scale
98     g = np.array([2.0 * p * q, 1.0 + p * p - q * q, 2.0 * q]) / scale
99     return f, g
100
101
102 def equinoctial(g: dict, r: np.ndarray) -> dict:
103     """Direct conversion for ellipses, with a stated hyperbolic extension.
104
105     h = e sin(0mega + omega), k = e cos(0mega + omega),
106     p = tan(i/2) sin(0mega), q = tan(i/2) cos(0mega).
107     L and F are true and eccentric longitude; lambda is mean longitude.
108     beta = 1 / (1 + sqrt(1 - h^2 - k^2)).
109     For hyperbolas, lambda = varpi + M_h, with varpi = atan2(h, k)
110     in [0, 2 pi), M_h = e sinh(F) - F, and no wrapping of lambda.
111     This extends the mean-longitude definition, not the elliptic F/beta formulas.
112     The posigrade convention is singular only at i = pi.
113     """
114     momentum = g["H"]
115     denominator = np.linalg.norm(momentum) + momentum[2]
116     if denominator <= 1e-12 or g["e"] == 1.0:
117         raise ValueError(
118             "A nonparabolic orbit with inclination below 180 deg is required."
119         )
120     p, q = momentum[0] / denominator, -momentum[1] / denominator
121     f, b = equinoctial_basis(p, q)
122     h, k = g["ev"] @ b, g["ev"] @ f
123     longitude = np.arctan2(r @ b, r @ f) % TAU
124     if g["e"] < 1.0:
125         beta = 1.0 / (1.0 + np.sqrt(1.0 - h * h - k * k))
126         eccentric_longitude = longitude + 2.0 * np.arctan2(
127             h * np.cos(longitude) - k * np.sin(longitude),
128             1.0 / beta + k * np.cos(longitude) + h * np.sin(longitude),
129         )
130         mean_longitude = (
131             eccentric_longitude
132             + h * np.cos(eccentric_longitude)
133             - k * np.sin(eccentric_longitude)
134         ) % TAU
135     else:
136         varpi = np.arctan2(h, k) % TAU

```

```

137     anomaly = hyperbolic_anomaly(g["e"], longitude - varpi)
138     mean_longitude = varpi + g["e"] * np.sinh(anomaly) - anomaly
139     return {
140         "a": g["a"],
141         "h": h,
142         "k": k,
143         "p": p,
144         "q": q,
145         "lambda": mean_longitude,
146         "L": longitude,
147     }
148
149
150 def reconstruct(eq: dict) -> tuple[np.ndarray, np.ndarray]:
151     """Invert elliptic elements or the unwrapped hyperbolic extension."""
152     a, h, k, p, q, longitude = (eq[key] for key in ("a", "h", "k", "p", "q", "lambda"))
153     eccentricity = np.hypot(h, k)
154     if a < 0.0 and eccentricity > 1.0:
155         varpi = np.arctan2(h, k) % TAU
156         mean_anomaly = longitude - varpi
157         # e sinh(F) - F is odd and strictly increasing for e > 1.
158         bound = max(1.0, np.arcsinh(abs(mean_anomaly) / eccentricity) + 1.0)
159         while eccentricity * np.sinh(bound) - bound < abs(mean_anomaly):
160             bound *= 2.0
161         anomaly = brentq(
162             lambda f: eccentricity * np.sinh(f) - f - mean_anomaly,
163             -bound,
164             bound,
165             xtol=1e-14,
166         )
167         scale = -a
168         beta = np.sqrt(eccentricity**2 - 1.0)
169         position = scale * np.array(
170             [eccentricity - np.cosh(anomaly), beta * np.sinh(anomaly)]
171         )
172         rate = np.sqrt(MU_EARTH / scale**3) / (eccentricity * np.cosh(anomaly) - 1.0)
173         velocity = scale * rate * np.array([-np.sinh(anomaly), beta * np.cosh(anomaly)])
174         f, b = equinoctial_basis(p, q)
175         # Perigee and transverse directions, expressed in the equinoctial basis.
176         basis = np.column_stack(
177             ((k * f + h * b) / eccentricity, (-h * f + k * b) / eccentricity)
178         )
179         return basis @ position, basis @ velocity
180     if a <= 0.0 or eccentricity >= 1.0:
181         raise ValueError(
182             "Semimajor axis and eccentricity must describe a nonparabolic conic."

```

```

183     )
184     anomaly = brentq(
185         lambda f: f + h * np.cos(f) - k * np.sin(f) - longitude,
186         longitude - eccentricity - 1e-12,
187         longitude + eccentricity + 1e-12,
188         xtol=1e-14,
189     )
190     beta = 1.0 / (1.0 + np.sqrt(1.0 - h * h - k * k))
191     transform = np.array(
192         [
193             [1.0 - beta * h * h, beta * h * k],
194             [beta * h * k, 1.0 - beta * k * k],
195         ]
196     )
197     position = a * (transform @ [np.cos(anomaly), np.sin(anomaly)] - [k, h])
198     rate = np.sqrt(MU_EARTH / a**3) / (1.0 - k * np.cos(anomaly) - h * np.sin(anomaly))
199     velocity = a * rate * (transform @ [-np.sin(anomaly), np.cos(anomaly)])
200     basis = np.column_stack(equinoctial_basis(p, q))
201     return basis @ position, basis @ velocity
202
203
204 def verify(
205     r: np.ndarray, v: np.ndarray, g: dict, eq: dict, coe: dict | None
206 ) -> list[str]:
207     """Check state reconstruction and independent classical identities."""
208     recovered_r, recovered_v = reconstruct(eq)
209     np.testing.assert_allclose(recovered_r, r, rtol=0.0, atol=1e-7)
210     np.testing.assert_allclose(recovered_v, v, rtol=0.0, atol=1e-10)
211     np.testing.assert_allclose(
212         np.dot(g["H"], g["H"]) / MU_EARTH,
213         g["a"] * (1.0 - g["e"] ** 2),
214         rtol=1e-12,
215     )
216     if coe is not None:
217         varpi = coe["Omega"] + coe["omega"]
218         expected = [
219             coe["e"] * np.sin(varpi),
220             coe["e"] * np.cos(varpi),
221             np.tan(coe["i"] / 2.0) * np.sin(coe["Omega"]),
222             np.tan(coe["i"] / 2.0) * np.cos(coe["Omega"]),
223         ]
224         np.testing.assert_allclose(
225             [eq[key] for key in ("h", "k", "p", "q")], expected, atol=1e-12
226         )
227         if g["e"] < 1.0:
228             delta = eq["lambda"] - varpi - coe["M"]

```

```

229         np.testing.assert_allclose(
230             [np.cos(delta), np.sin(delta)], [1.0, 0.0], atol=1e-12
231         )
232     else:
233         np.testing.assert_allclose(
234             eq["lambda"], varpi % TAU + coe["M_h"], atol=1e-12
235         )
236     # Independent state-vector relation for the signed hyperbolic anomaly.
237     np.testing.assert_allclose(
238         np.sinh(coe["F"]),
239         (r @ v) / (g["e"] * np.sqrt(MU_EARTH * -g["a"])),
240         atol=1e-12,
241     )
242     return [
243         f"position reconstruction error = {np.linalg.norm(recovered_r - r):.3e} km",
244         f"velocity reconstruction error = {np.linalg.norm(recovered_v - v):.3e} km/s",
245     ]
246
247
248 def element_lines(elements: dict) -> list[str]:
249     """Format numerical elements; the report reads these key/value lines."""
250     angles = {
251         "i",
252         "Omega",
253         "omega",
254         "theta",
255         "theta_signed",
256         "E",
257         "M",
258         "lambda",
259         "L",
260         "varpi",
261     }
262     lines = []
263     for key, value in elements.items():
264         # if key == "lambda" and elements["a"] < 0.0:
265         #     lines.append(f"lambda = {value:.10f} rad")
266         # elif key in angles:
267         if key in angles:
268             lines.append(f"{key} = {np.degrees(value):.6f} deg")
269         elif key == "a":
270             lines.append(f"a = {value:.6f} km")
271         else:
272             lines.append(f"{key} = {value:.10f}")
273     return lines
274

```

```

275
276 def solve_state(number: str, r: list[float], v: list[float], both: bool = True) -> None:
277     """Save classical/equinotial results and Cartesian diagnostics."""
278     r, v = np.array(r, dtype=float), np.array(v, dtype=float)
279     g = geometry(r, v)
280     eq = equinotial(g, r)
281     coe = classical(g, r) if both else None
282     diagnostics = [
283         f"r = {g['r']:.6f} km",
284         f"vr = {g['vr']:.6f} km/s",
285         f"energy = {g['energy']:.9f} km^2/s^2",
286         f"H vector = {np.array2string(g['H'], precision=6)} km^2/s",
287         f"N vector = {np.array2string(g['N'], precision=6)} km^2/s",
288         f"e vector = {np.array2string(g['ev'], precision=10)}",
289     ]
290     checks = verify(r, v, g, eq, coe)
291     if both:
292         assert coe is not None
293         write_output(number + "a", element_lines(coe) + diagnostics)
294         write_output(number + "b", element_lines(eq) + checks)
295     else:
296         write_output(
297             number, element_lines(eq) + [f"e = {g['e']:.10f}"] + diagnostics + checks
298         )
299
300
301 def problem_04() -> None:
302     """Curtis 4.6: choose the inbound branch of true anomaly."""
303     heading("p04")
304     r = np.array([-6000.0, -1000.0, -5000.0])
305     ev = np.array([0.4, 0.5, 0.6])
306     cosine = r @ ev / (np.linalg.norm(r) * np.linalg.norm(ev))
307     principal = np.arccos(np.clip(cosine, -1.0, 1.0))
308     theta = TAU - principal
309     assert np.sin(theta) < 0.0
310     write_output(
311         "04",
312         [
313             f"r = {np.linalg.norm(r):.6f} km",
314             f"e = {np.linalg.norm(ev):.10f}",
315             f"e dot r = {ev @ r:.6f} km",
316             f"cos theta = {cosine:.10f}",
317             f"principal = {np.degrees(principal):.6f} deg",
318             f"theta = {np.degrees(theta):.6f} deg",
319         ],
320     )

```

```

321
322
323 def problem_05() -> None:
324     """Curtis 4.7: inward radial motion fixes the orbit normal's sign."""
325     heading("p05")
326     r = np.array([-6600.0, -1300.0, -5200.0])
327     ev = np.array([-0.4, -0.5, -0.6])
328     # e x r = e r sin(theta) H_hat; inbound means sin(theta) < 0.
329     cross = np.cross(r, ev)
330     normal = cross / np.linalg.norm(cross)
331     inclination = np.arctan2(np.linalg.norm(normal[:2]), normal[2])
332     # Reconstruct a compatible velocity to verify the selected direction.
333     semilatus = np.linalg.norm(r) + ev @ r
334     momentum = np.sqrt(MU_EARTH * semilatus)
335     velocity = MU_EARTH / momentum * np.cross(normal, ev + r / np.linalg.norm(r))
336     assert r @ velocity < 0.0
337     np.testing.assert_allclose(np.cross(r, velocity) / momentum, normal, atol=1e-12)
338     write_output(
339         "05",
340         [
341             f"r cross e = {np.array2string(cross, precision=6)} km",
342             f"cross magnitude = {np.linalg.norm(cross):.6f} km",
343             f"cos i = {normal[2]:.10f}",
344             f"i = {np.degrees(inclination):.6f} deg",
345             f"compatible radial velocity = {r @ velocity / np.linalg.norm(r):.6f} km/s",
346         ],
347     )
348
349
350 def main() -> None:
351     #####
352     # p01 #
353     #####
354     heading("p01")
355     solve_state("01", [2500, 16000, 4000], [-3, -1, 5])
356
357     #####
358     # p02 #
359     #####
360     heading("p02")
361     solve_state("02", [0, 0, -13000], [4, 5, 6])
362
363     #####
364     # p03 #
365     #####
366     heading("p03")

```

```
367     solve_state("03", [6500, -7500, -2500], [4, 3, -3])
368
369     #####
370     # p04 #
371     #####
372     problem_04()
373
374     #####
375     # p05 #
376     #####
377     problem_05()
378
379     #####
380     # p06 #
381     #####
382     heading("p06")
383     solve_state("06a", [3698.3455, -34232.4473, 0], [2.1482, -1.4944, 0], both=False)
384     solve_state("06b", [-21082.0848, 36515.242, 0], [-2.6627, -1.5373, 0], both=False)
385     solve_state(
386         "06c",
387         [-15843.4562, -2247.7766, 21200.4627],
388         [1.4544, -3.5188, 0.7138],
389         both=False,
390     )
391
392
393 if __name__ == "__main__":
394     main()
```